

Compose 기초

1 Jetpack Compose란

오래된 레거시

아주 오래된 레거시

1.

Jetpack Compose란

15년 전의 모바일 환경



HTC Dream



Android 1.0

15년 전(2008년)에 만들어진 레거시 UI 코드와 API

아주 오래된 레거시

안드로이드 타임라인

1.

Jetpack Compose란

1. 1.0 (2008년 9월 23일)
2. 1.1 Petit Four (2009년 2월 9일)
3. 1.5 Cup Cake (2009년 4월 27일)
4. 1.6 Donut (2009년 9월 15일)
5. 2.0 Eclair (2009년 10월 27일)
6. 2.2 Froyo (2010년 5월 20일)
7. 2.3 Gingerbread (2010년 12월 6일)
8. 3.0 Honeycomb (2011년 2월 22일)
9. 4.0 Ice Cream Sandwich (2011년 10월 18일)
- 10.4.1 Jelly Bean (2012년 7월 9일)
- 11.4.4 Kitkat (2013년 10월 31일)
- 12.5.0 Lollipop (2014년 11월 4일)

- 13.6.0 Marshmallow (2015년 10월 2일)
- 14.7.0 Nougat (2016년 8월 22일)
- 15.8.0 Oreo (2017년 8월 21일)
- 16.9 Pie (2018년 8월 6일)
- 17.10 (Q) (2019년 9월 3일)
- 18.11 (R) (2020년 9월 8일)
- 19.12 (S) (2021년 10월 4일)
20. 13 Tiramisu (2022년 8월 15일)

아주 오래된 레거시

가장 크게 UI가 바뀐 버전

1.

Jetpack Compose란

1. 1.0 (2008년 9월 23일)
2. 1.1 Petit Four (2009년 2월 9일)
3. 1.5 Cup Cake (2009년 4월 27일)
4. 1.6 Donut (2009년 9월 15일)
5. 2.0 Eclair (2009년 10월 27일)
6. 2.2 Froyo (2010년 5월 20일)
7. 2.3 Gingerbread (2010년 12월 6일)
8. 3.0 Honeycomb (2011년 2월 22일) **Holo**
9. 4.0 Ice Cream Sandwich (2011년 10월 18일)
10. 4.1 Jelly Bean (2012년 7월 9일)
11. 4.4 Kitkat (2013년 10월 31일)
12. 5.0 Lollipop (2014년 11월 4일) **Material**

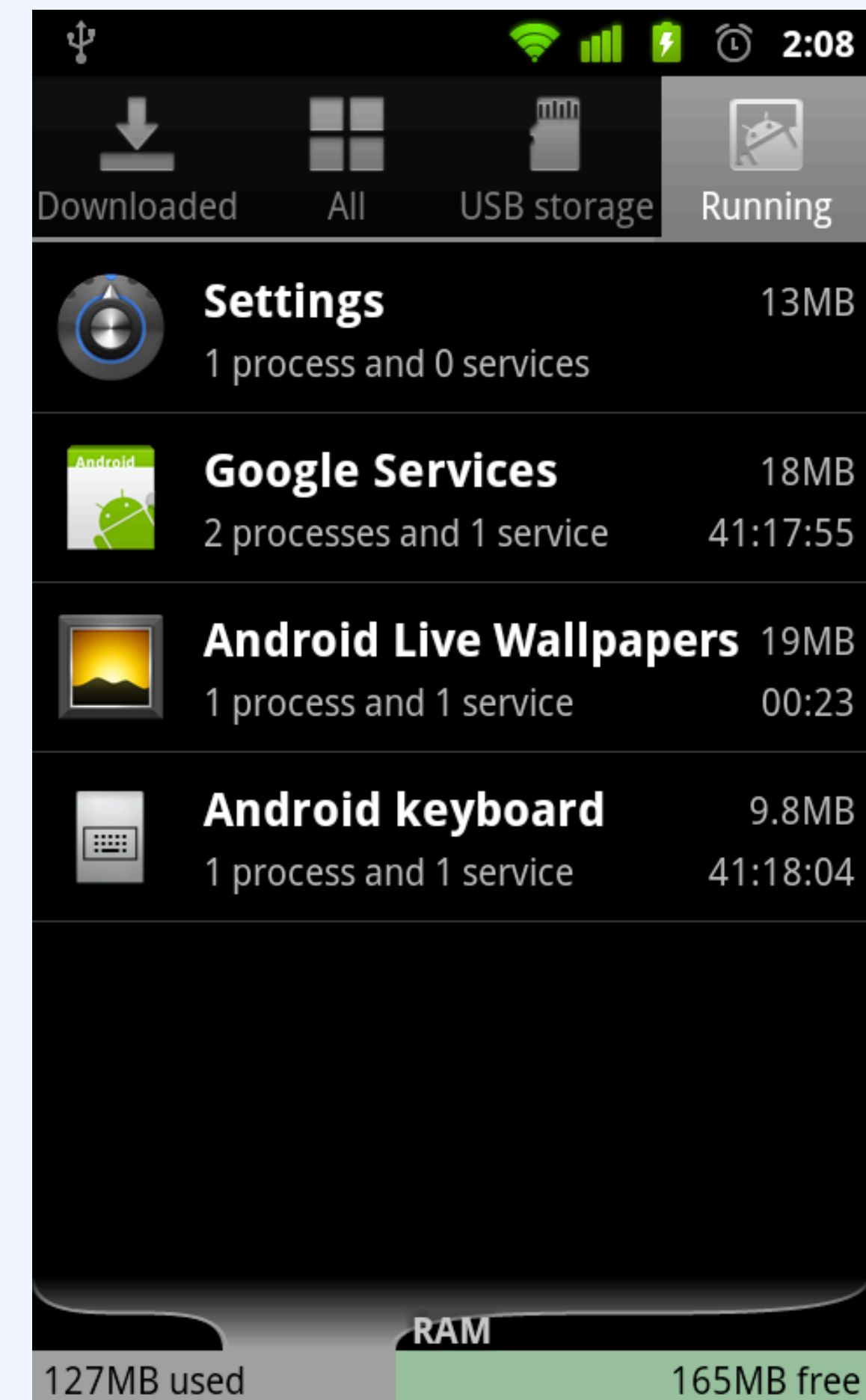
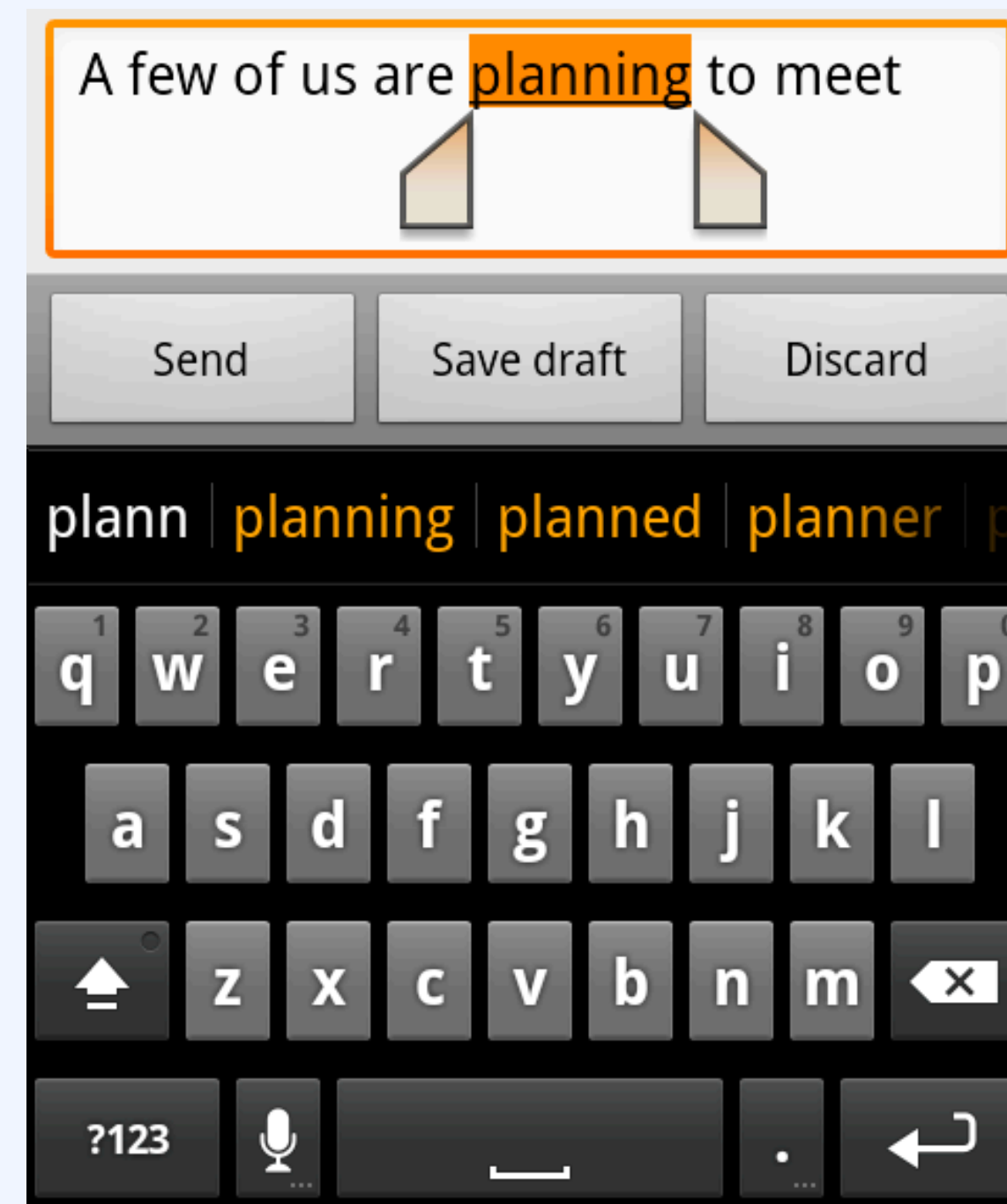
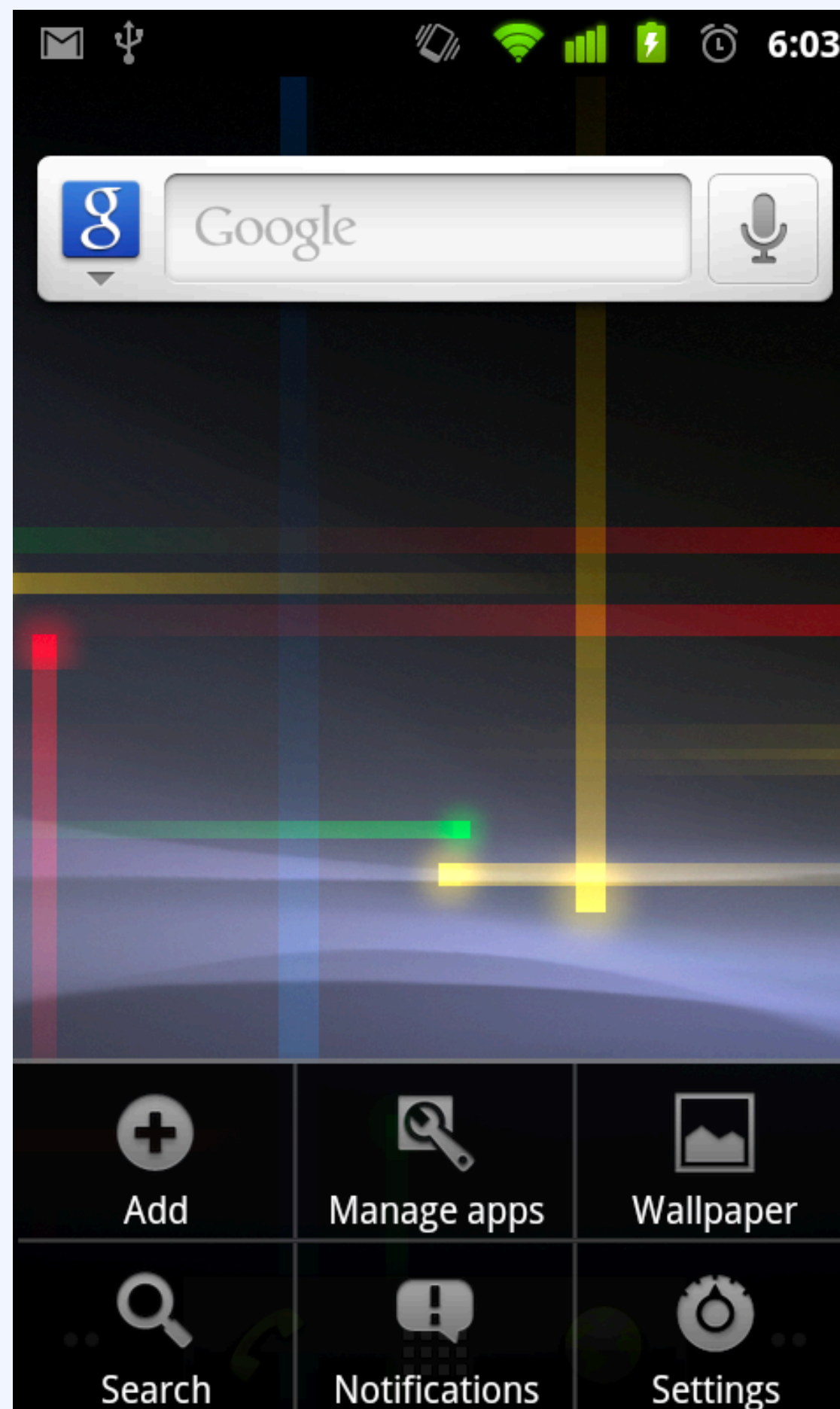
13. 6.0 Marshmallow (2015년 10월 2일)
14. 7.0 Nougat (2016년 8월 22일)
15. 8.0 Oreo (2017년 8월 21일)
16. 9 Pie (2018년 8월 6일)
17. 10 (Q) (2019년 9월 3일)
18. 11 (R) (2020년 9월 8일)
19. 12 (S) (2021년 10월 4일)
20. 13 Tiramisu (2022년 8월 15일)

아주 오래된 레거시

Pre-Honeycomb

1.
Jetpack Compose란

3.0이 태블릿 전용이었고 2.X 진저브레드의 생명력이 강해 킷캣(4.4) 전 까지 유지

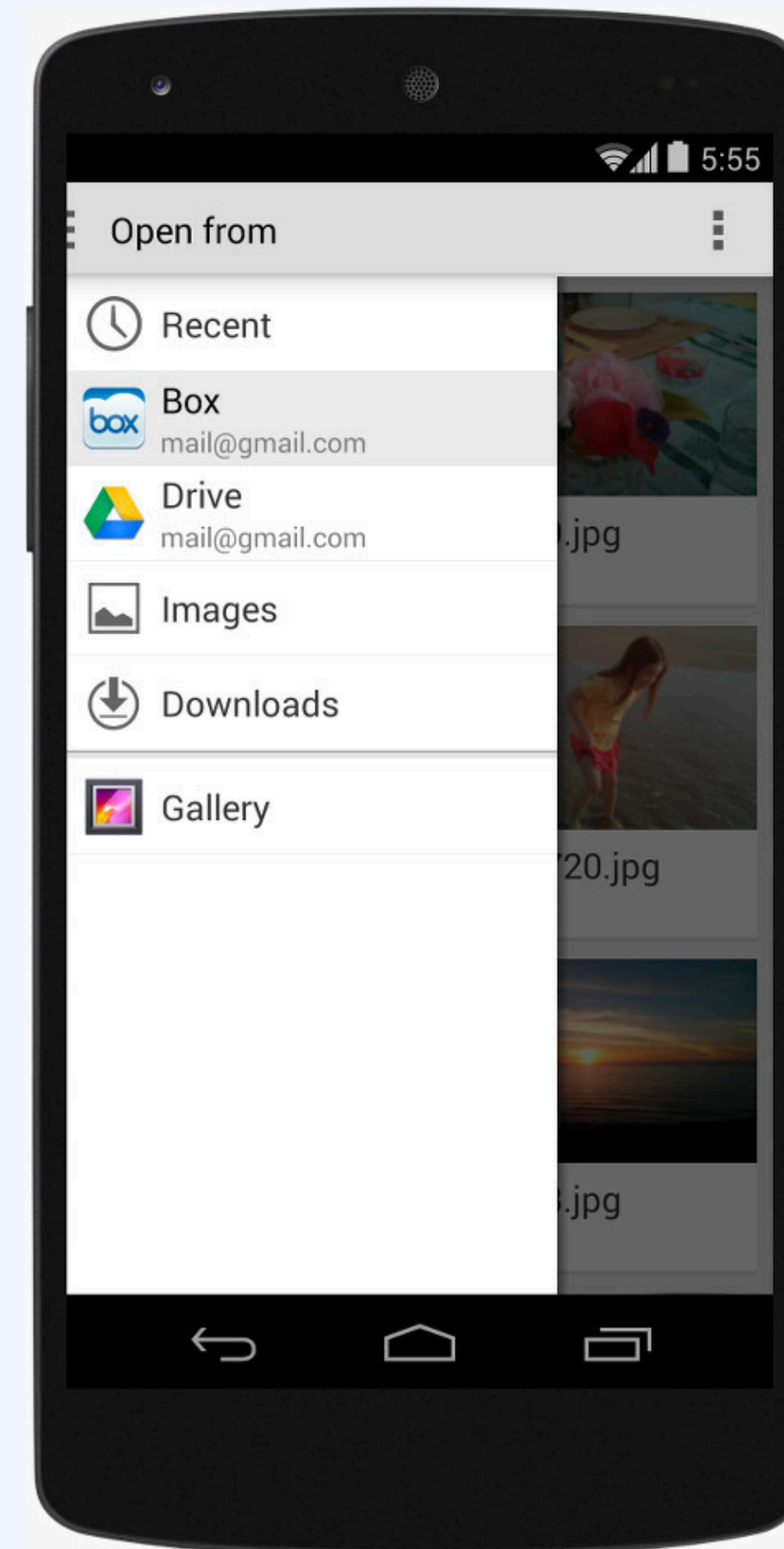
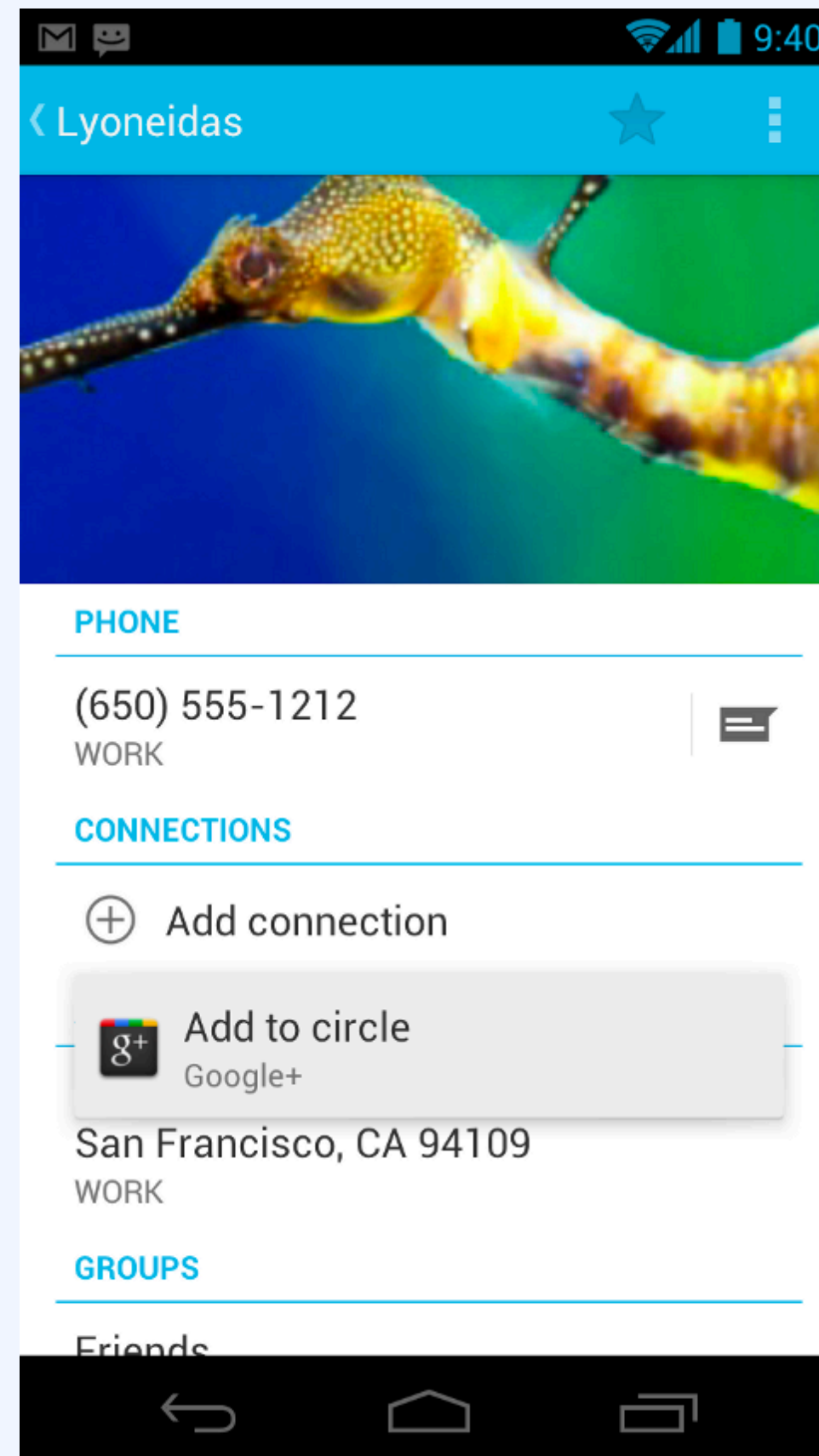
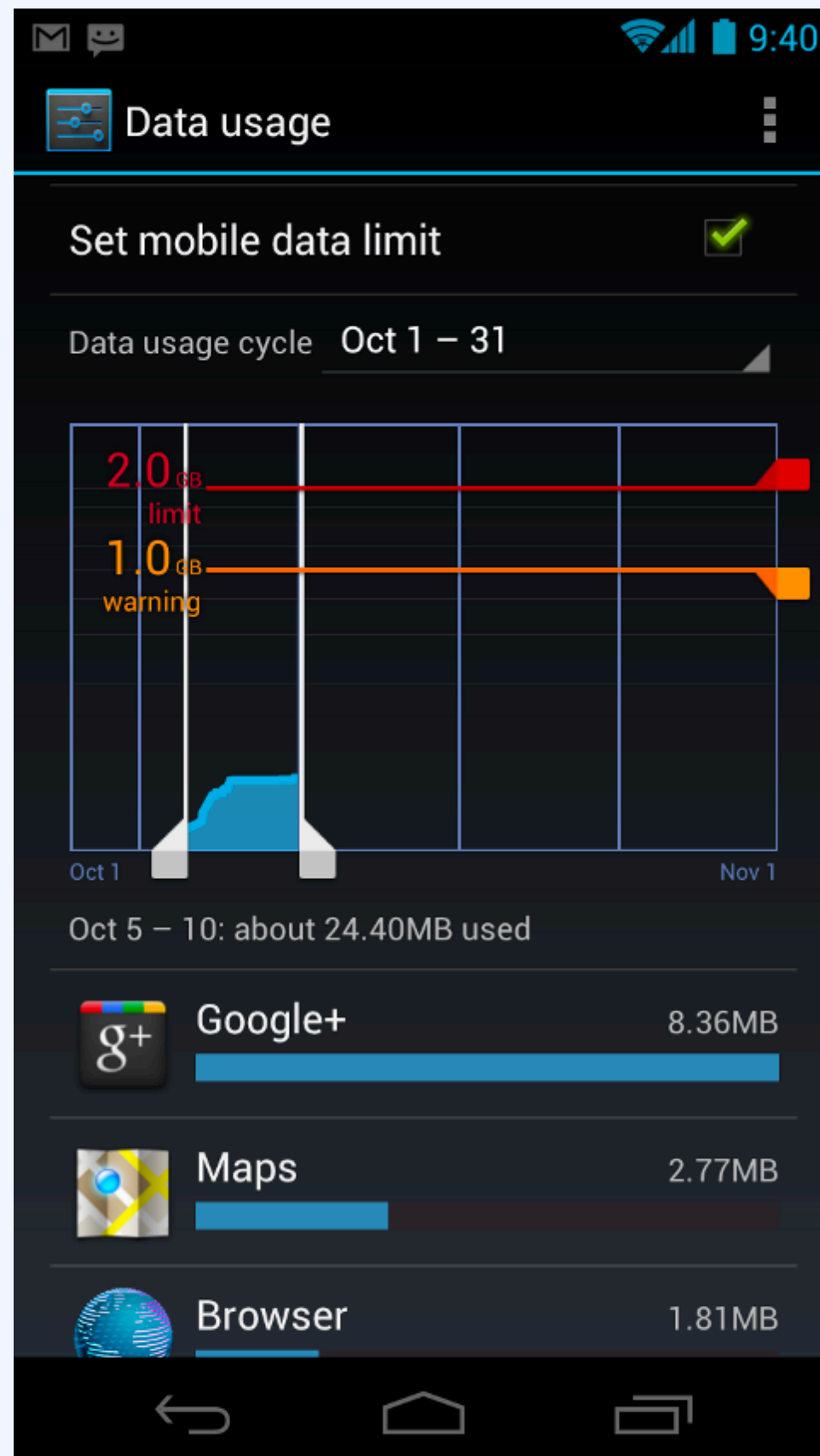


아주 오래된 레거시

Holo (2012)

1.
Jetpack Compose란

스퀘어모픽이 유행할 때 플랫 디자인을 시도. 3.0이 타블렛 전용이었기 때문에 4.0부터 퍼지기 시작. 끈질긴 수명의 진저브레드가 버터 4.4 킷킷에 이르러 대중화.

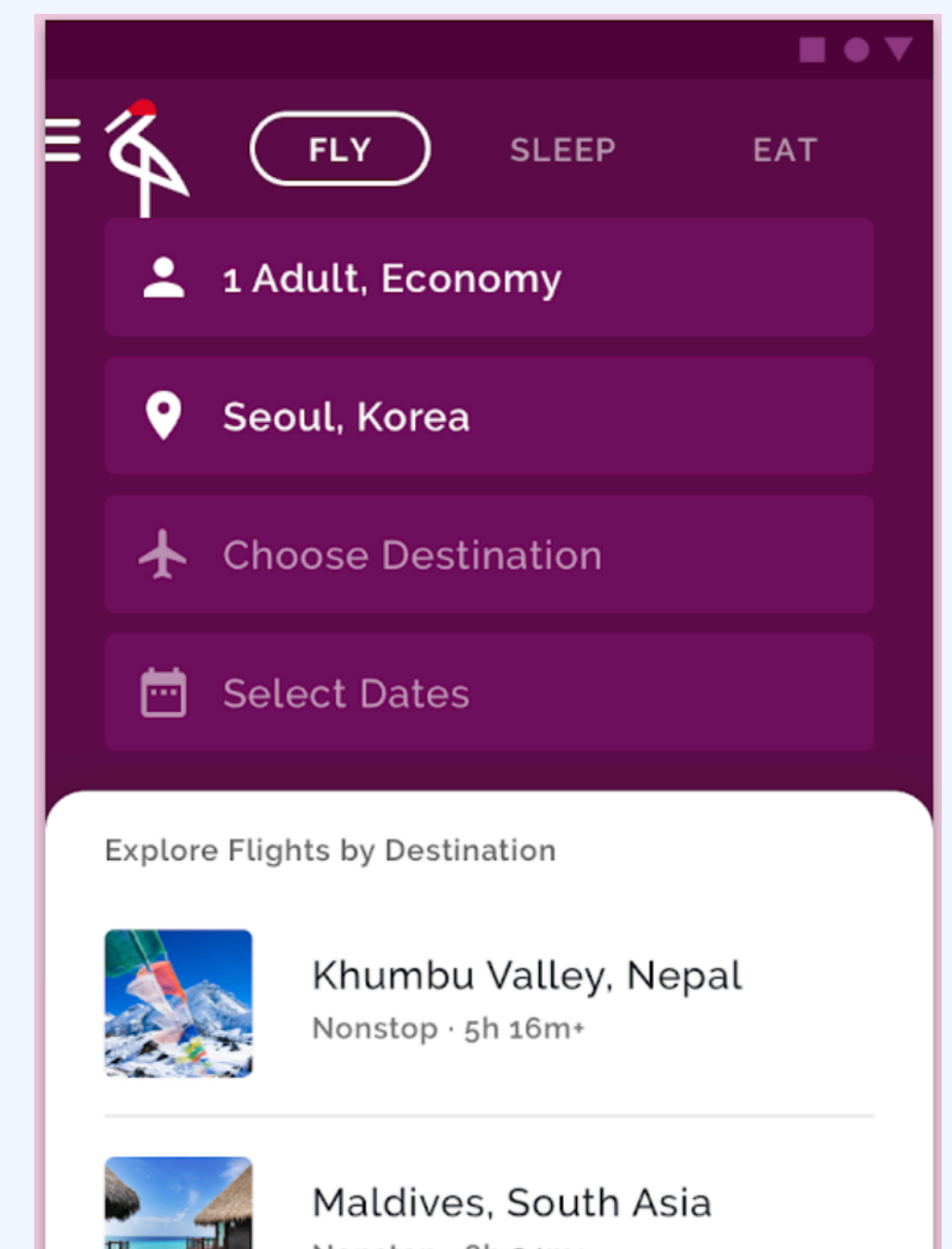
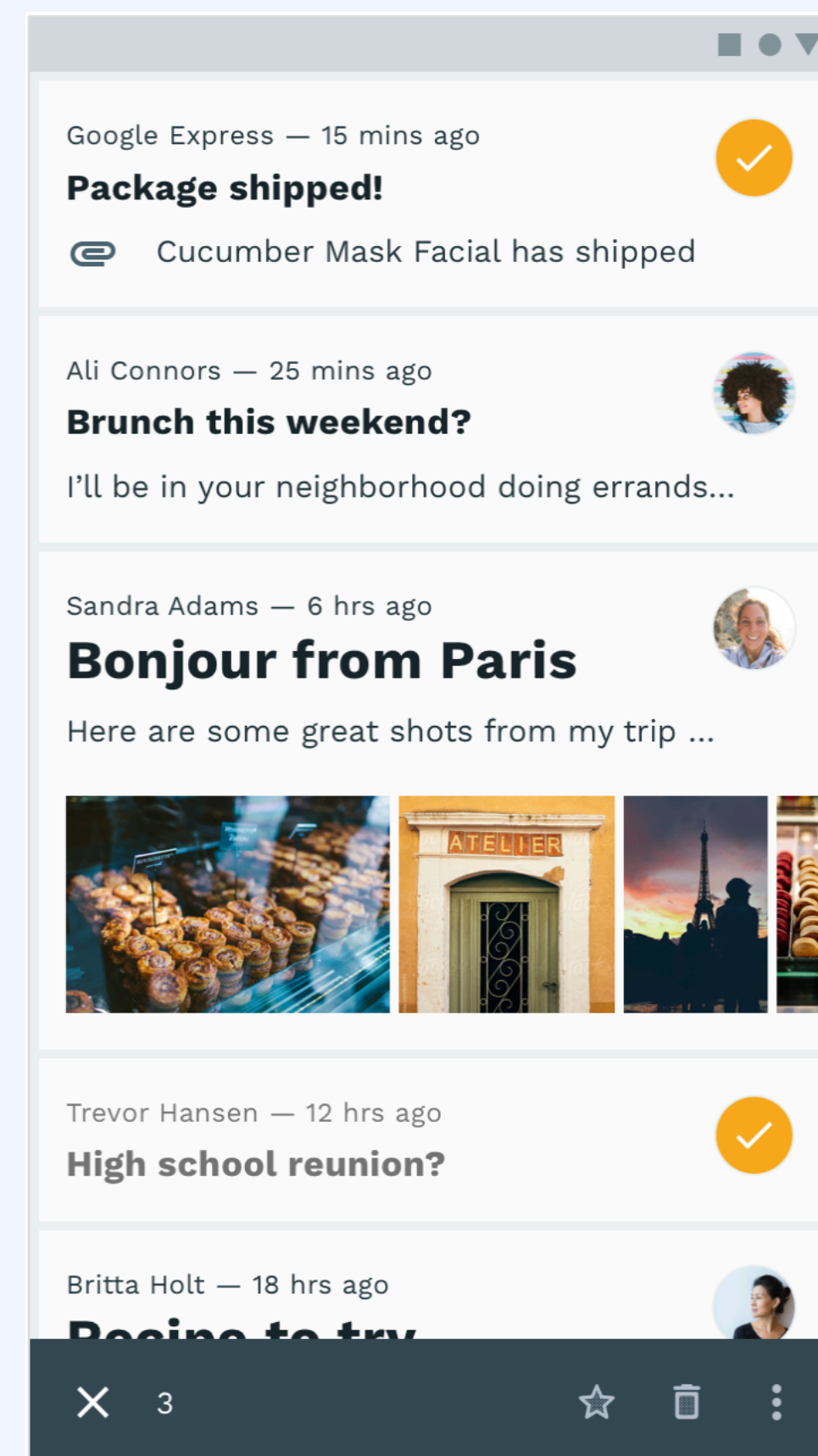
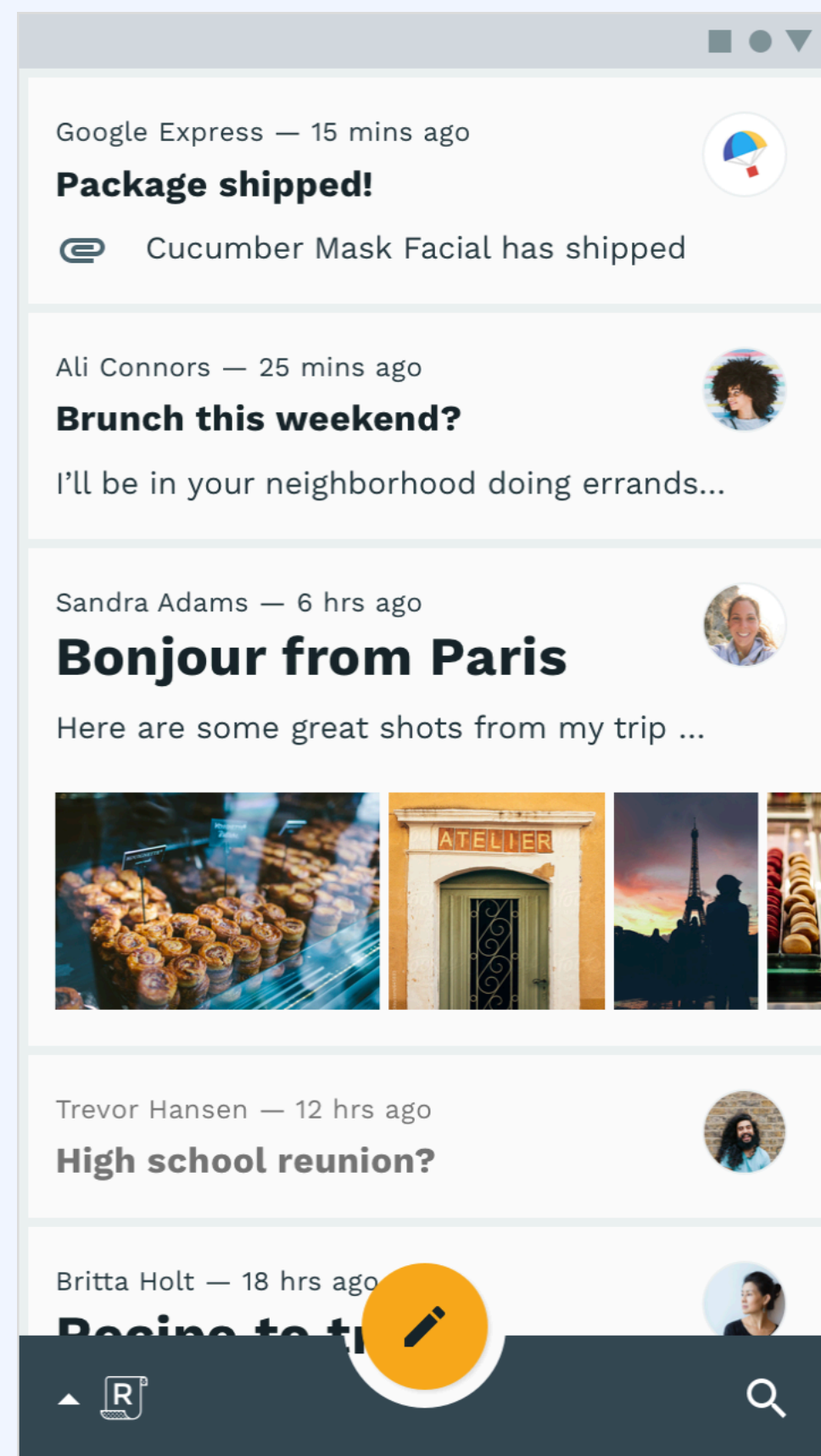
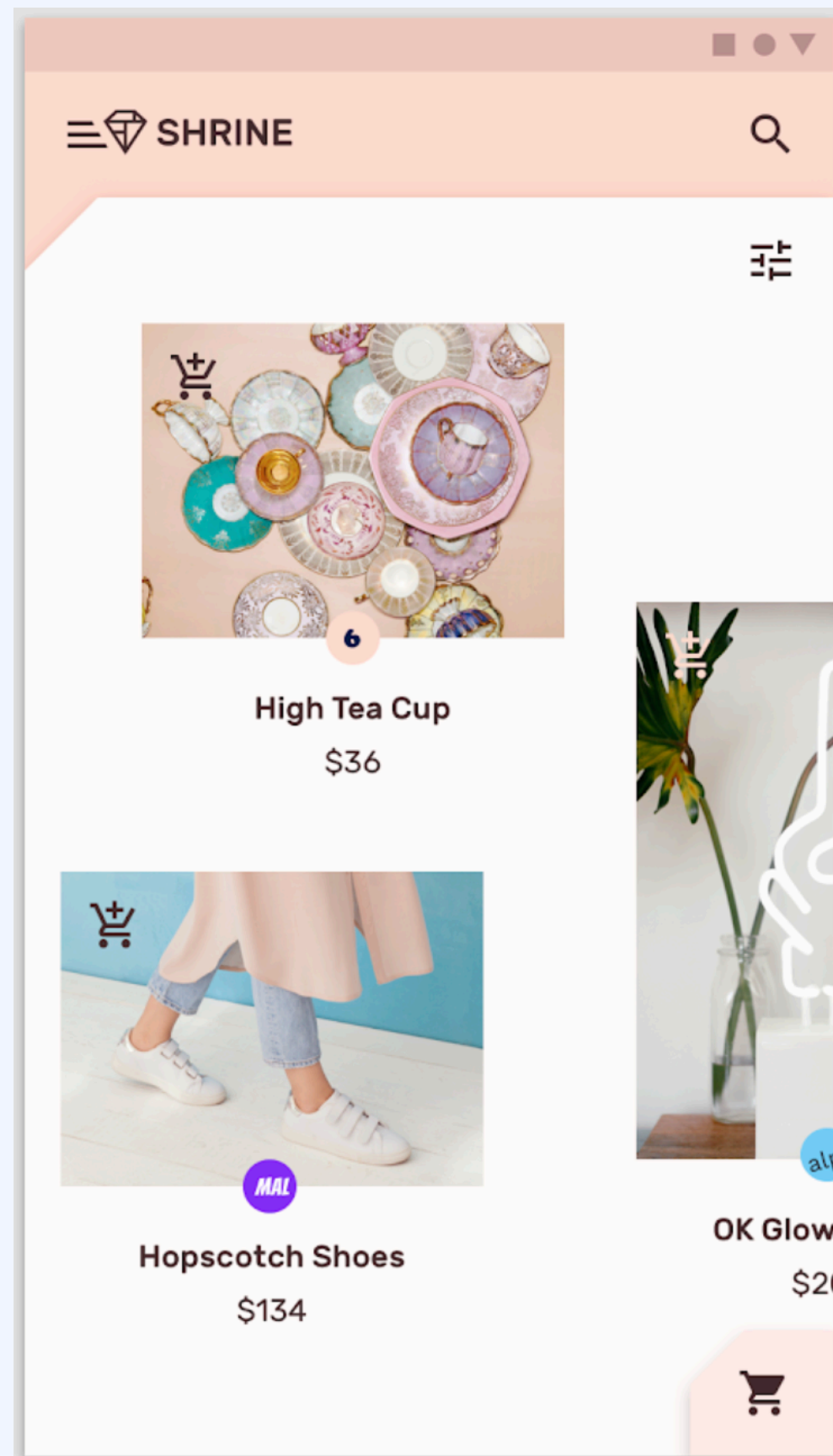


아주 오래된 레거시

Material Design

1.
Jetpack Compose란

마티아스 두아르테가 만든 구글의 새로운 디자인 언어. 머터리얼 디자인 (2014)
Holo, Kennedy 등의 여러 디자인 언어의 대안으로 등장.



아주 오래된 레거시

버전 별 차이를 매꾸기 위한 써드파티 라이브러리

1.
Jetpack Compose란

진저브레드 단말이 오래 살아남았고 액션바를 대응하기 위해 액션바 설록과 같은 라이브러리가 오래동안 사용되었음. 구글의 AppCompat v21이 나와서야 해결.

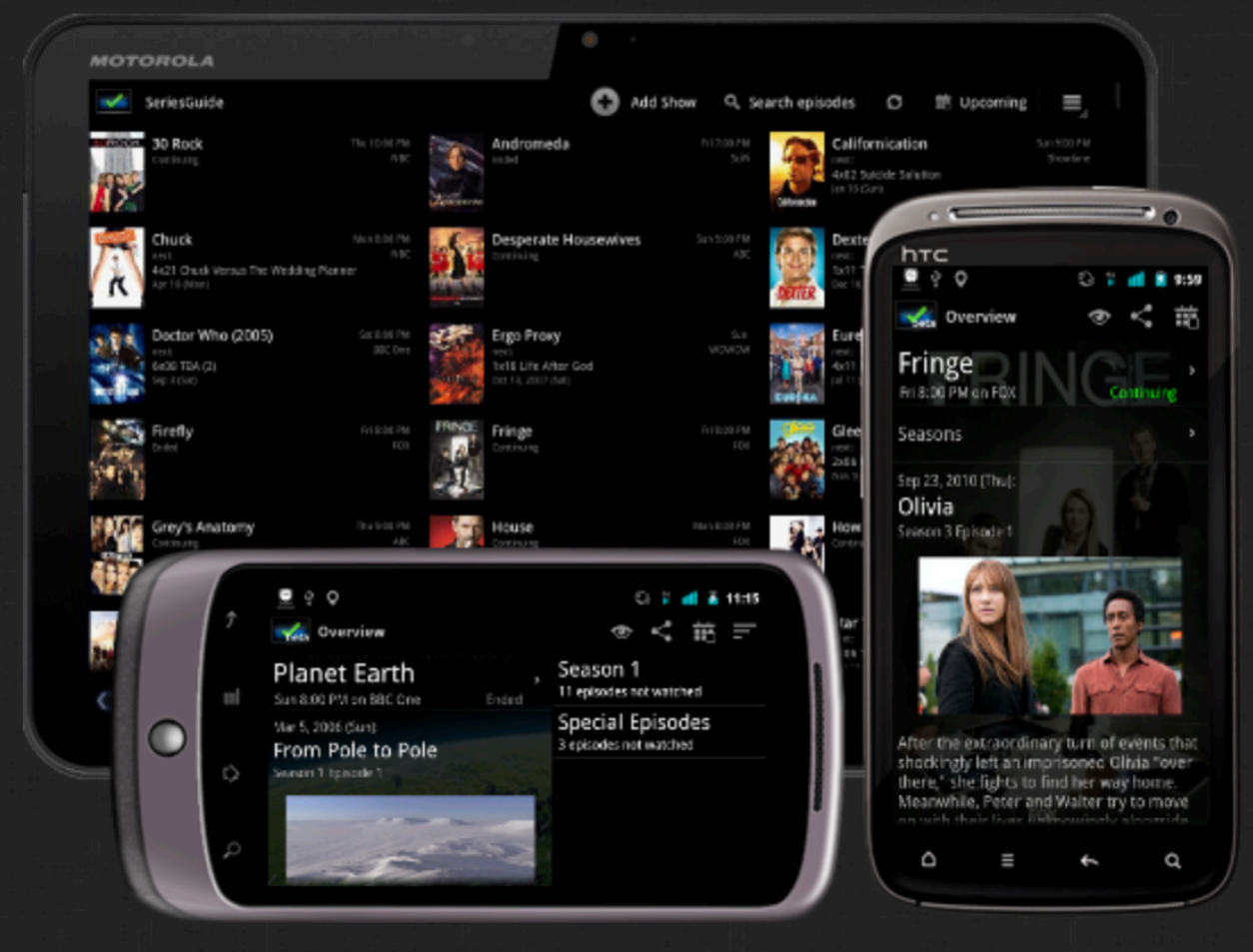
ActionBarSherlock

ActionBarSherlock is an extension of the [support library](#) designed to facilitate the use of the action bar design pattern across all versions of Android with a single API.



The library will automatically use the native action bar when appropriate or will automatically wrap a custom implementation around your layouts. This allows you to easily develop an application with an action bar for every version of Android from 2.x and up.

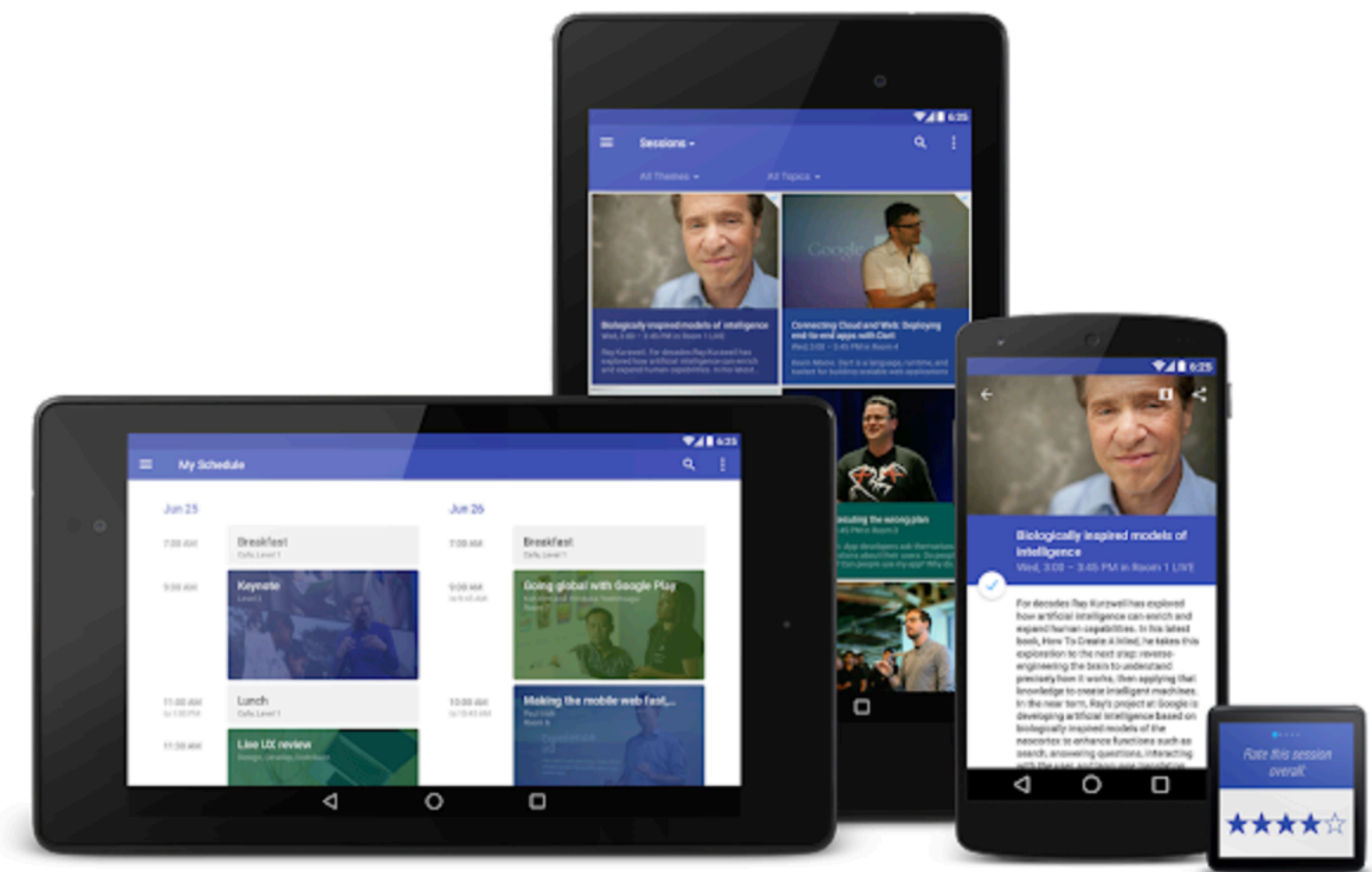
Download v4.4.0 : [Zip](#) [Tarball](#) [More...](#)



AppCompat v21 — Material Design for Pre-Lollipop Devices!

22 October 2014

By Chris Banes, Android Developer Relations



The [Android 5.0 SDK](#) was released last Friday, featuring new UI widgets and material design, our visual language focused on good design. To enable you to bring your latest designs to older Android platforms we have expanded our support libraries, including a major update to [AppCompat](#), as well as new [RecyclerView](#), [CardView](#) and [Palette](#) libraries.

In this post we'll take a look at what's new in AppCompat and how you can use it to support

아주 오래된 레거시

세월이 느껴지는 API

1.
Jetpack Compose란

오랜 세월을 겪어 오며 다양한 버전에서 Deprecated가 발생.

int	STATUS_BAR_VISIBLE This constant was deprecated in API level 15. Use SYSTEM_UI_FLAG_VISIBLE instead.
int	SYSTEM_UI_FLAG_FULLSCREEN This constant was deprecated in API level 30. Use WindowInsetsController#hide(int) with Type#statusBar instead.
int	SYSTEM_UI_FLAG_HIDE_NAVIGATION This constant was deprecated in API level 30. Use WindowInsetsController#hide(int) with Type#navigationBar instead.
int	SYSTEM_UI_FLAG_IMMERSIVE This constant was deprecated in API level 30. Use WindowInsetsController#BEHAVIOR_DEFAULT instead.

아주 오래된 레거시

자바 레거시

1.
Jetpack Compose란

자바로 시작했기 때문에 코틀린에 최적화되어 있지 않음.

```
public boolean onKeyDown(int keyCode, KeyEvent event) {
    if (keyCode == KeyEvent.KEYCODE_DPAD_CENTER) {
        // When the user center presses, let them pick a contact.
        startActivityForResult(
            new Intent(Intent.ACTION_PICK,
                new Uri("content://contacts")),
            PICK_CONTACT_REQUEST);
        return true;
    }
    return false;
}
```


아주 오래된 레거시

계속 바뀌는 개발 방식

1.
Jetpack Compose란

! This class was deprecated in API level 30.
Use the standard `java.util.concurrent` or [Kotlin concurrency utilities](#) instead.

AsyncTask was intended to enable proper and easy use of the UI thread. However, the most common use case was for integrating into UI, and that would cause Context leaks, missed callbacks, or crashes on configuration changes. It also has inconsistent behavior on different versions of the platform, swallows exceptions from `doInBackground`, and does not provide much utility over using `Executor`s directly.

AsyncTask

Observable

```
public abstract class Observable
extends Object
```

Observable

RxJava: Reactive Extensions for the JVM

Snapshot passing codecov 100% maven central 3.1.5 Contribute with Gitpod

RxJava is a Java VM implementation of [Reactive Extensions](#): a library for composing asynchronous and event-based programs by using observable sequences.

It extends the [observer pattern](#) to support sequences of data/events and adds operators that allow you to compose sequences together declaratively while abstracting away concerns about things like low-level threading, synchronization, thread-safety and concurrent data structures.

RxJava

LiveData  Artifact: [androidx.lifecycle:lifecycle-livedata-core](#) [View Source](#)

[Kotlin](#) | [Java](#)

```
public abstract class LiveData<T>
```

Known direct subclasses
[MutableLiveData](#), [SliceLiveData](#), [CachedSliceLiveData](#)

Known indirect subclasses
[MediatorLiveData](#)

LiveData is a data holder class that can be observed within a given lifecycle. This means that an [Observer](#) can be added in a pair with a [LifecycleOwner](#), and this observer will be notified about modifications of the wrapped data only if the paired LifecycleOwner is in active state. LifecycleOwner is considered as active, if its state is [STARTED](#) or [RESUMED](#). An observer added via [observeForever](#) is considered as always active and thus will be always notified about modifications. For those observers, you should manually call [removeObserver](#).

LiveData

Android Jetpack

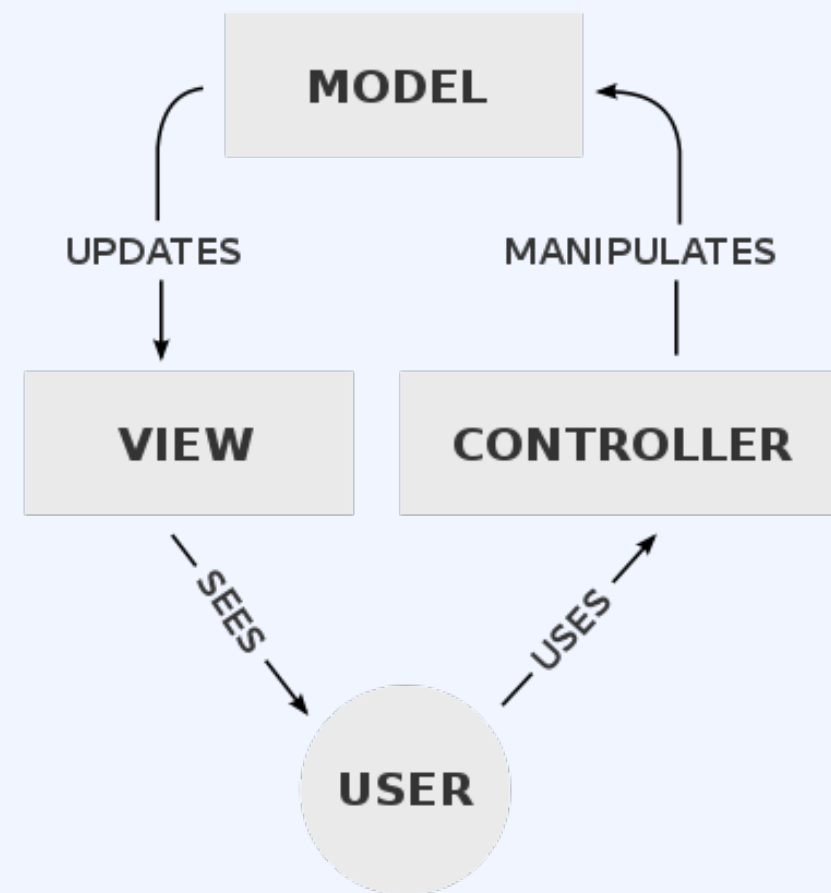
Jetpack은 개발자가 중요한 코드에 집중할 수 있도록 권장사항 준수, 상용구 코드 감소, 여러 Android 버전과 기기에서 일관되게 작동하는 코드 작성을 돕는 라이브러리 묶음입니다.

Jetpack

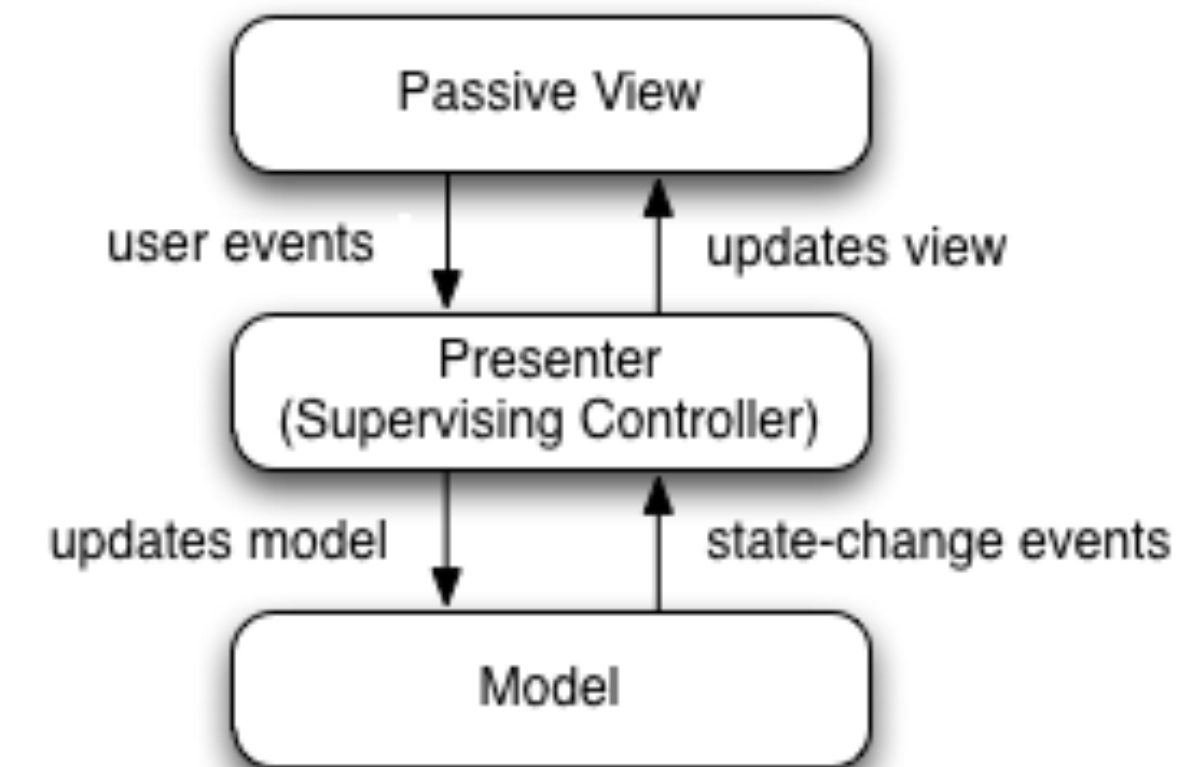
아주 오래된 레거시

변하는 아키텍처

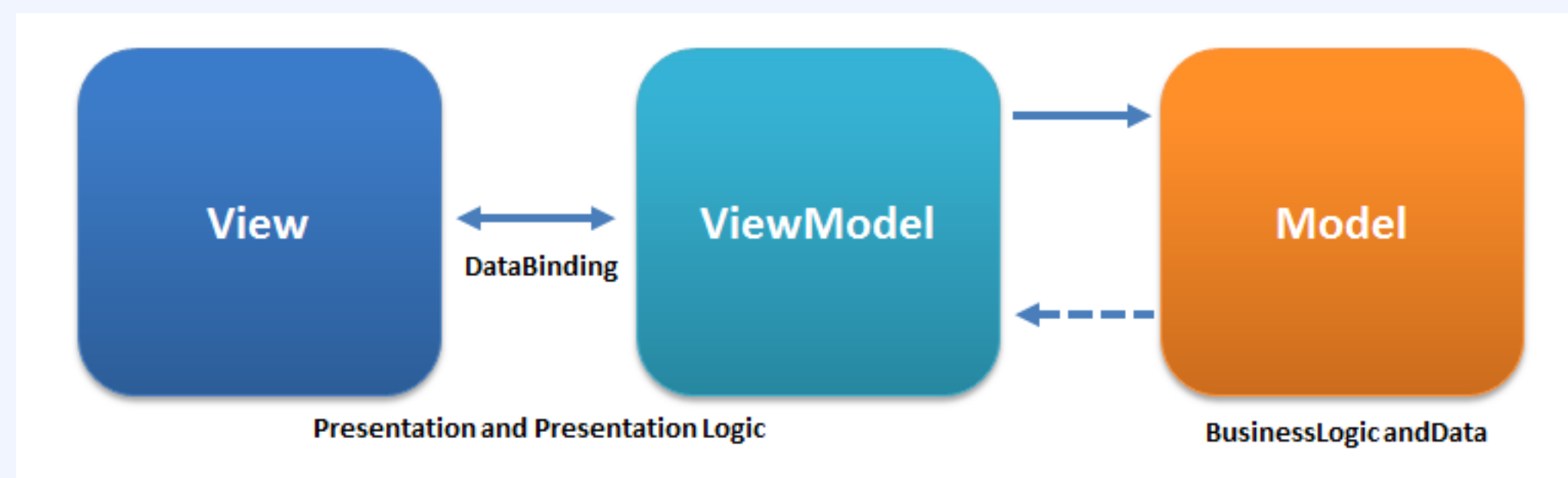
1.
Jetpack Compose란



Model-View-Controller



Model-View-Presenter

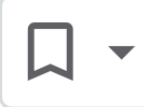


Model-View-ViewModel

데이터 결합 라이브러리



[Android Jetpack](#)의 구성요소.



데이터 결합 라이브러리는 프로그래매틱 방식이 아니라 선언적 형식으로 레이아웃의 UI 구성 요소를 앱의 데이터 소스와 결합할 수 있는 지원 라이브러리입니다.

레이아웃은 흔히 UI 프레임워크 메서드를 호출하는 코드가 포함된 활동에서 정의됩니다. 예를 들어 아래 코드는 `findViewById()` 를 호출하여 `TextView` 위젯을 찾아 `viewModel` 변수의 `userName` 속성에 결합합니다.

뷰 결합



이 페이지의 내용

설정 안내

사용

활동에서 뷰 결합 사용

프래그먼트에서 뷰 결합 사용

`findViewById`와의 차이점

데이터 결합과 비교

뷰 결합 기능을 사용하면 뷰와 상호작용하는 코드를 쉽게 작성할 수 있습니다. 모듈에서 사용 설정된 뷰 결합은 모듈에 있는 각 XML 레이아웃 파일의 **결합 클래스**를 생성합니다. 바인딩 클래스의 인스턴스에는 상응하는 레이아웃에 ID가 있는 모든 뷰의 직접 참조가 포함됩니다.

대부분의 경우 뷰 결합이 `findViewById` 를 대체합니다.